

WEB DEVELOPMENT USING PYTHON AND DJANGO

Module 1 Programs:

Aim: In this program, using SQL queries we manipulate by inserting, deleting, retrieving data of a particular record.

1.Connecting to SQL Databases:

a. Sqlite/ oracle/ mysql

The Python programming language has powerful features for database programming. Python supports various databases like SQLite, MySQL, Oracle, Sybase, PostgreSQL, etc. Python also supports Data Definition Language (DDL), Data Manipulation Language (DML) and Data Query Statements. The Python standard for database interfaces is the Python DB-API. Most Python database interfaces adhere to this standard.

>>Connect To Database (sqlite):

Following Python code shows how to connect to an existing database. If the database does not exist, then it will be created and finally a database object will be returned.

>> Code:

```
import sqlite3

conn = sqlite3.connect('Company.db')

print('Opened database successfully')
```

2. Data manipulation using python:

>>Create a Table

Following Python program will be used to create a table in the previously created database.

>>Code:

```
#!/usr/bin/python

import sqlite3

conn = sqlite3.connect('Company.db')

print ('Opened database successfully')

conn.execute('CREATE TABLE COMPANY

(ID INT PRIMARY KEY NOT NULL,

NAME TEXT NOT NULL,

AGE INT NOT NULL,

ADDRESS CHAR(50),

SALARY REAL);')

Print('created successfully')

conn.close()
```

>>Insert Operation

Following Python program shows how to create records in the COMPANY table created in the above example.

>>Code:

```
#!/usr/bin/python
```

```
Import sqlite3
```

```
Conn= sqlite3.connect('company.db')
```

```
Print('opened database successfully');
```

```
Conn.execute('INSERT INTO COMPANY(ID,NAME,AGE,ADDRESS,  
SALARY)/ VALUES(1,'Abhi',20,'Bangalore',25000.00)');
```

```
Conn.execute('INSERT INTO COMPANY(ID,NAME,AGE,ADDRESS,  
SALARY)/ VALUES(2,'Sampath',21,'Chennai',35000.00)');
```

```
Conn.execute('INSERT INTO COMPANY(ID,NAME,AGE,ADDRESS,  
SALARY)/ VALUES(3,'Manjula',35,'Kannur',40000.00)');
```

```
Conn.execute('INSERT INTO COMPANY(ID,NAME,AGE,ADDRESS,  
SALARY)/ VALUES(4,'Abhirami',25,'Bangalore',30000.00)');
```

>>Select Operation

Following Python program shows how to fetch and display records from the COMPANY table created in the above example.

>>Code:

```
#!/usr/bin/python

import sqlite3

conn = sqlite3.connect('company.db')

print('Opened database successfully')

cursor = conn.execute('SELECT id, name, address, salary from COMPANY')

for row in cursor:

    print("ID = ", row[0])

    print("NAME = ", row[1])

    print("ADDRESS = ", row[2])

    print("SALARY = ", row[3], "\n")

print("Operation done successfully")

conn.close()
```

>>Select Operation

// you can also try below for select

```
import sqlite3
```

```
# Connect to the database
```

```
conn = sqlite3.connect('company.db')
```

```
# Fetch and display data
```

```
cursor = conn.execute('SELECT * FROM COMPANY')
```

```
print('Data in Company table')
```

```
for row in cursor:
```

```
    print(row)
```

```
conn.close()
```

>>Update Operation

Following Python code shows how to use UPDATE statement to update any record and then fetch and display the updated records from the COMPANY table.

>>Code:

```
#!/usr/bin/python

import sqlite3

conn = sqlite3.connect('company.db')

print("Opened database successfully")

conn.execute("UPDATE COMPANY set SALARY = 28000.00 where ID = 1")

conn.commit

print("Total number of rows updated", conn.total_changes)

cursor = conn.execute("SELECT id, name, address, salary from COMPANY")

for row in cursor:

    print("ID = ", row[0])

    print("NAME = ", row[1])

    print("ADDRESS = ", row[2])

    print("SALARY = ", row[3], "\n")

print("Operation done successfully")

conn.close()
```

>>Delete Operation

Following Python code shows how to use DELETE statement to delete any record and then fetch and display the remaining records from the COMPANY table.

>>code:

```
#!/usr/bin/python

import sqlite3

conn = sqlite3.connect('company.db')

print("Opened database successfully")

conn.execute('DELETE from COMPANY where ID = 2;')

conn.commit()

print('Total number of rows deleted :', conn.total_changes)

cursor = conn.execute("SELECT id, name, address, salary from COMPANY")

for row in cursor:

    print("ID = ", row[0])

    print("NAME = ", row[1])

    print("ADDRESS = ", row[2])

    print("SALARY = ", row[3], "\n")

print("Operation done successfully")

conn.close()
```

Result: We have successfully connected using DDL and manipulated data using DML queries. We have performed operations like insertion, deletion etc.,

MODULE-2

MAPIT.PY WITH THE WEBBROWSER MODULE

Aim: Web scraping is the process of using programs to automatically download and process web content. It is commonly used for data collection, research, and automation, with search engines like Google relying on it to index web pages. In an internet-driven world, web scraping helps automate tasks and extract data efficiently. Python simplifies this process with powerful modules, making it an essential skill for handling repetitive online tasks and large-scale data extraction.

- Web browser Comes with Python and opens a browser to a specific page.
- requests Downloads files and web pages from the internet.
- bs4 Parses HTML, the format that web pages are written in.
- selenium Launches and controls a web browser.
- The selenium module is able to fill in forms and simulate mouse clicks in this browser.

Example 1. mapIt.py with the web browser Module

The web browser module's `open()` function can launch a new browser to a specified URL. Enter the following into the interactive shell:

```
>> import webbrowser  
>> web browser.open('https://www.cmr.edu.in/')
```

Example 2: Google map

The mapIt.py program automates the task of opening a Google Maps location, saving time by reducing manual steps.

>>Code:

```
#!/ python3

# mapIt.py - Launches a map in the browser using an address from the # command
line or clipboard.

import webbrowser

import sys

import pyperclip

import urllib.parse

if len(sys.argv)>1:

    #Get address from command line

    #print(sys.argv[1:])

    address=''.join(sys.argv[1: ])

    print(address)

else:

    #Get address from clipboard

    address=pyperclip.paste()

    # URL encode the address properly

    encoded_address = urllib.parse.quote(address)

    # Open Google Maps

    webbrowser.open("https://www.google.com/maps/place/"+address)
```

FOR MANUAL INPUT:

```
import webbrowser

import urllib.parse

# Ask the user for manual input
address = input("Enter the location: ").strip()

# URL encode the address
encoded_address = urllib.parse.quote_plus(address)

# Open Google Maps with the correct query format
webbrowser.open(f'https://www.google.com/maps/search/?api=1&query={encoded_address}')
```

“I’M FEELING LUCKY” GOOGLE SEARCH

```
#!/ python3

# lucky.py - Opens several Google search results.

import requests, sys, webbrowser, bs4

print('Googling...') # display text while downloading the Google page
res=requests.get('http://google.com/search?q='+' '.join(sys.argv[1:]))
res.raise_for_status()

#Retrieve top search result links

soup=bs4.BeautifulSoup(res.text, features='html.parser')

linkElems = soup.select('div#main > div > div > div > a')

numOpen = min(5, len(linkElems))

for i in range(numOpen):

    webbrowser.open('http://google.com' + linkElems[i].get('href'))
```

OR

```
import sys

import sys

from selenium import webdriver

from selenium.webdriver.common.by import By

import time

# Get the search term from command-line arguments

sterm = ' '.join(sys.argv[1:])
```

```
# Use the Chrome WebDriver (you may need to install it first)

driver = webdriver.Chrome()

# Navigate to Google and perform the search

driver.get('https://www.google.com/search?q={sterm}')

# Wait for the page to load

time.sleep(5)

# Find the first search result link using the new method

search_results = driver.find_elements(By.CSS_SELECTOR, 'h3 a')

if search_results:

    # Open the first result

    driver.get(search_results[0].get_attribute('href'))

else:

    print('No search results found!')

# Close the browser after a short wait

time.sleep(5)

driver.quit()
```

Result: In this program, we have successfully executed the code for mapIt.py program automates the task of opening a Google Maps location, saving time by reducing manual steps and then opening the website using requests library.

MODULE-3

Aim: In this program, we will be performing operation to download XKCD comics, controlling browser using selenium and to read data using urllib.

3.1 DOWNLOADING ALL XKCD COMICS:

```
#!/ python3

# downloadXkcd.py - Downloads every single XKCD comic.

import requests, os, bs4

url = 'https://xkcd.com/' # starting url

n = input("Input the comic number: ")

# Append the user comic number in the url

url += n

os.makedirs('xkcd', exist_ok=True) # store comics in ./xkcd

while not url.endswith('#'):

    # Download the page.

    print('Downloading page %s...' % url)

    res = requests.get(url)

    res.raise_for_status()

    soup = bs4.BeautifulSoup(res.text, features="html.parser")

    # Find the URL of the comic image.

    comicElem = soup.select('#comic img')

    if comicElem == []:

        print('Could not find comic image.')
```

```
else:

    comicUrl = 'http:' + comicElem[0].get('src')

    # Download the image.

    print('Downloading image %s...' % (comicUrl))

    res = requests.get(comicUrl)

    res.raise_for_status()

    # Save the image to ./xkcd

    imageFile = open(os.path.join('xkcd', os.path.basename(comicUrl)), 'wb')

    for chunk in res.iter_content(100000):

        imageFile.write(chunk)

    imageFile.close()

    # Get the Prev button's url.

    prevLink = soup.select('a[rel="prev"]')[0]

    url = 'http://xkcd.com' + prevLink.get('href')

print('Done.')
```

3.2 Controlling the Browser with the selenium Module:

Objective: This program automates a browser using selenium to perform a search on the Python official website.

>>>**code**

```
from selenium import webdriver

from selenium.webdriver.common.by import By

from selenium.webdriver.common.keys import Keys

from time import sleep

# Use Options() to handle ChromeDriver setup properly

options = webdriver.ChromeOptions()

# Start Chrome WebDriver

driver = webdriver.Chrome(options=options)

# Open Python website

driver.get("https://www.python.org")

print(driver.title)

# Find the search bar

search_bar = driver.find_element(By.NAME, "q")

search_bar.clear()

# Enter search query

search_bar.send_keys("getting started with python")

sleep(2)

search_bar.send_keys(Keys.RETURN)
```

```
# Print the current URL
```

```
print(driver.current_url)
```

```
# Wait before closing
```

```
sleep(15)
```

```
driver.quit()
```


3.3 Read website data using Urllib library:

Objective: The objective of this task is to demonstrate the use of the urllib library to retrieve, process, and extract data from web pages in Python. The urllib library simplifies web scraping tasks by allowing users to send HTTP requests and handle responses seamlessly. The goal is to teach how to:

1. Read data from a web page.
2. Perform simple operations like counting word frequency.
3. Extract specific content using regular expressions.

i. Read data from a web page.

>>code

```
import urllib.request
```

```
import urllib.parse
```

```
import re
```

```
# 2. Search
```

```
url = 'https://analyticsindiamag.com/'
```

```
values = {
```

```
    's': 'Web Scraping',
```

```
    'submit': 'search'
```

```
}
```

3. Parse

```
data = urllib.parse.urlencode(values)
```

```
data = data.encode('utf-8')
```

```
req = urllib.request.Request(url, data)
```

```
resp = urllib.request.urlopen(req)
```

```
respData = resp.read()
```

4. Extract using regular expressions

```
document = re.findall(r'<p>(.*?)</p>', str(respData))
```

for line in document:

```
    print(line)
```

ii. Perform simple operations like counting word frequency.

>>code

```
import urllib.request, urllib.parse, urllib.error

fhand = urllib.request.urlopen('http://data.pr4e.org/romeo.txt')

counts = dict()

for line in fhand:

    words = line.decode().split()

    for word in words:

        counts[word] = counts.get(word, 0) + 1

print(counts)
```

iii. Extract specific content using regular expressions.

>>>code

```
import urllib.request
```

```
import re
```

```
# Fetch the webpage
```

```
url = "https://analyticsindiamag.com/"
```

```
response = urllib.request.urlopen(url)
```

```
html = response.read().decode('utf-8')
```

```
# Extract the title using regex
```

```
title = re.findall(r"<title>(.*?)</title>", html)
```

```
print("Title:", title[0] if title else "No title found")
```

```
# Extract all paragraphs
```

```
paragraphs = re.findall(r"<p>(.*?)</p>", html)
```

```
print("\nParagraphs:")
```

```
for para in paragraphs:
```

```
    print(para)
```

Result: By performing this experiment, we have acquired the knowledge about the process downloading images using program,controlling browser and to read website using urllib.

MODULE-4

Aim: In this experiment, we will be creating website using Django and performing adding images operation and adding login/user using “just works”.

1. Create website using Django framework:

>>Code

• Creating the App

Install Django (if not installed)

```
pip install django
```

Create Django project

```
django-admin startproject mywebsite
```

Navigate to the project directory

```
cd mywebsite
```

Create a Django app

```
python manage.py startapp myapp
```

• URL Mapping

myapp/urls.py

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [  
    path("", views.home, name='home'),  
    # Add more paths as needed  
]
```

mywebsite/urls.py

```
from django.contrib import admin  
from django.urls import path, include
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path("", include('myapp.urls')),  
]
```

• Error Handling

Django handles errors by default (e.g., 404, 500).

To customize error pages, create 404.html, 500.html, etc., in your directory.

• Running the Server

```
python manage.py runserver
```

Open your browser and go to <http://127.0.0.1:8000/> to see your site.

- **Connecting to Databases**

In **mywebsite/settings.py**, update the **DATABASES** section:

```
DATABASES = {  
  
    'default': {  
  
        'ENGINE': 'django.db.backends.sqlite3',  
  
        'NAME': BASE_DIR / 'db.sqlite3',  
  
    }  
}
```

- **Creating HTML Files**

myapp/templates/home.html

```
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>  
  
    <meta charset="UTF-8">  
  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  
    <title>Welcome to My Website</title>  
  
</head>  
  
<body>  
  
    <h1>Welcome to My Website</h1>  
  
    <p>This is the home page content.</p>  
  
      
  
</body>  
  
</html>
```

2. Adding Images and Media

Update mywebsite/settings.py

```
import os

# Add the following at the bottom of the file

MEDIA_URL = '/media/'

MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

Update mywebsite/urls.py

```
from django.conf import settings

from django.conf.urls.static import static

# Add this to the bottom of your urlpatterns list

urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)


```


3. Add Login/Logout/User Registration to Website Using “Just Works”

myapp/urls.py:

```
from django.contrib.auth import views as auth_views

urlpatterns += [

    path('login/', auth_views.LoginView.as_view(), name='login'),

    path('logout/', auth_views.LogoutView.as_view(), name='logout'),

    path('register/', views.register, name='register'),

]
```

myapp/views.py:

```
from django.shortcuts import render, redirect

from django.contrib.auth.forms import UserCreationForm

def register(request):

    if request.method == 'POST':

        form = UserCreationForm(request.POST)

        if form.is_valid():

            form.save()

            return redirect('login')

    else:

        form = UserCreationForm()

    return render(request, 'registration/register.html', {'form': form})
```

Result: In this module 4 program,with the help of django framework we have created a website, added images and added login,logout,user registration using “just works”.